Curso: Ciência da Computação

Disciplina de Computação Gráfica

Transformações Geométricas

Aula 03

Professor: André Flores dos Santos

Santa Maria – RS 2026

SUMÁRIO

01 INTRODUÇÃO

02 OBJETIVO

03 OPERAÇÕES ALGÉBRICAS

04 TRANSLAÇÃO

05 ROTAÇÃO

06 ESCALA

Transformações Geométricas

- Objetivo:
  - **manipular o conteúdo** de uma cena;
- Como?
  - pelo **movimento** da **câmera** ou dos **objetos** que estão presentes na cena;
- Tipos de transformações:
  - **orientação**, **tamanho** e **formato** de um objeto;
- Essas mudanças podem ser cumulativas!

Transformações Geométricas

- **Transformações afim:**
  - Mantém o paralelismo de retas.
  - Preservam os ângulos;

- **Principais:**
  - **Translação**, **escala** e **rotação**.

- Como representar?
  - **Operações algébricas**
  - **Matrizes**.

Operações Algébricas

- Podem se tornar caras computacionalmente;
- Porquê matrizes são mais interessantes?
  - Podem realizar as transformações e **combiná-las de forma mais eficiente.**
  - Mais eficientes no armazenamento de figuras presentes num sistema de coordenadas.

Translação

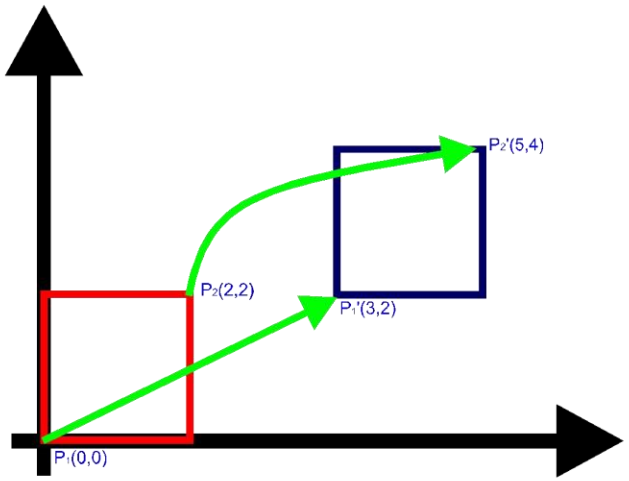
- Define a **posição** de um objeto no universo;
- A partir de coordenadas de um ponto (x_o,y_o), soma-se a ele os valores de translação T_x e T_y, obtendo-se assim o ponto (x_u,y_u), que representa o ponto posicionado no universo;
- Para tanto aplicam-se as seguinte equações:

$$x_u = x_o + T_x$$

$$y_u = y_o + T_y$$

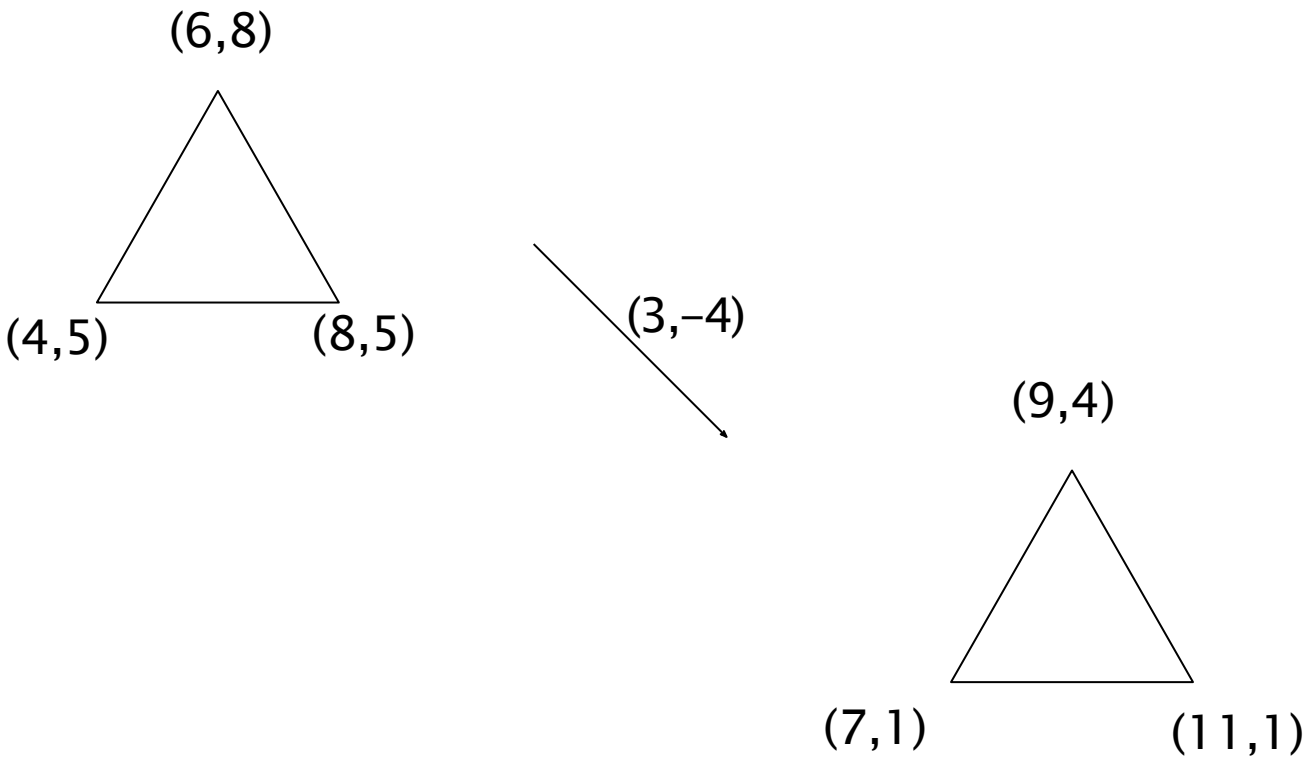
Translação

- Exemplo:
  - $P_1(0,0)$ e $P_2(2,2)$
  - $Tx = 3$ e $Ty = 2$
    - $P_1'(3,2)$ e $P_2'(5,4)$



Translação

- Exemplo 2:



Translação

- Também representada na forma matricial através da **soma de matrizes**:

$$P_o = \begin{bmatrix} x_o \\ y_o \end{bmatrix} \qquad P_u = \begin{bmatrix} x_u \\ y_u \end{bmatrix} \qquad T = \begin{bmatrix} T_x \\ T_Y \end{bmatrix}$$

$$P_u = P_o + T$$

$$P_u = \begin{bmatrix} x_o \\ y_o \end{bmatrix} + \begin{bmatrix} T_x \\ T_Y \end{bmatrix}$$

$$\boxed{P_u = \begin{bmatrix} x_o + T_x \\ y_o + T_Y \end{bmatrix}}$$

Translação

Vamos aplicar essa teoria na prática, através de um algoritmo em Python para a operação de translação para os pontos dos exemplos anteriores, **slide 7**. Exercício 01.

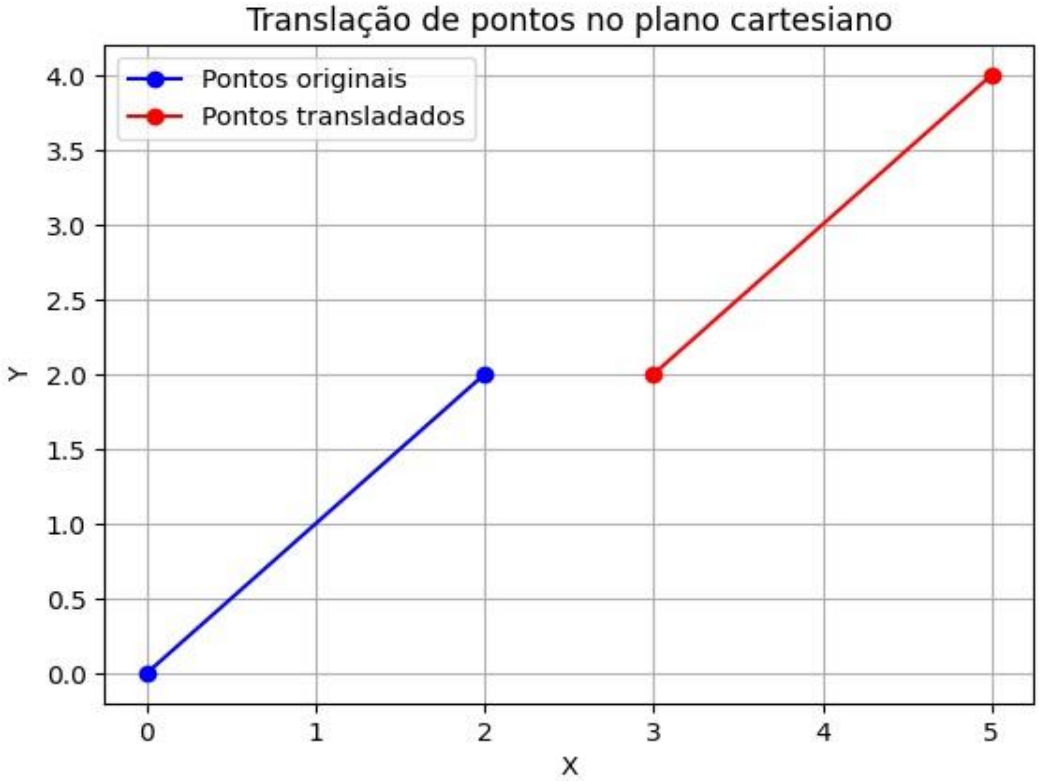
```python
import numpy as np
import matplotlib.pyplot as plt

# Função para calcular a translação dos pontos
def translacao(pontos, Tx, Ty):
    pontos_transladados = []
    for ponto in pontos:
        x_u = ponto[0] + Tx
        y_u = ponto[1] + Ty
        pontos_transladados.append((x_u, y_u))
    return pontos_transladados

# Pontos originais
p1 = (0, 0)
p2 = (2, 2)

# Vetor de translação
Tx = 3
Ty = 2

# Calcular a translação dos pontos
pontos_transladados = translacao([p1, p2], Tx, Ty)
```

```python
# Plotar os pontos originais e os pontos transladados
plt.plot([p1[0], p2[0]], [p1[1], p2[1]], 'bo-', label='Pontos originais’)

# Plotar os pontos transladados da mesma forma
# Pegando o X do primeiro [0][0] e o X do segundo [1][0]
lista_x = [ pontos_transladados[0][0], pontos_transladados[1][0] ]
# Pegando o Y do primeiro [0][1] e o Y do segundo [1][1]
lista_y = [ pontos_transladados[0][1], pontos_transladados[1][1] ]
plt.plot(lista_x, lista_y, 'ro-', label='Pontos transladados')

# Definir os limites dos eixos
plt.xlim(-5, 7)  # Define o limite do eixo X de 0 a 10
plt.ylim(-5, 7)  # Define o limite do eixo Y de 0 a 10

# Configurações do gráfico
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Translação de pontos no plano cartesiano')
plt.grid(True)
plt.legend()


# Mostrar o gráfico
plt.show()
```

Translação

Vamos aplicar essa teoria na prática, através de um algoritmo em Python para a operação de translação para os pontos dos exemplos anteriores, **slide 7**.

Gráfico gerado:



Translação

Vamos aplicar essa teoria na prática, através de um algoritmo em Python para a operação de translação para os pontos dos exemplos anteriores, **slide 8**. Exercício 02.

Agora complete o algoritmo anterior para o exemplo do Slide 8, que tem 3 pontos, ao invés de 2 apenas.

p1(6,8)
p2(4,5)
p3(8,5)

(tx, ty) = (3,-4)

p1'(?,?) = (9,4)
p2'(?,?) = (7,1)
p3'(?,?) = (11,1)

OBS: Todos exercícios devem ser enviados através de um link do seu repositório no github.

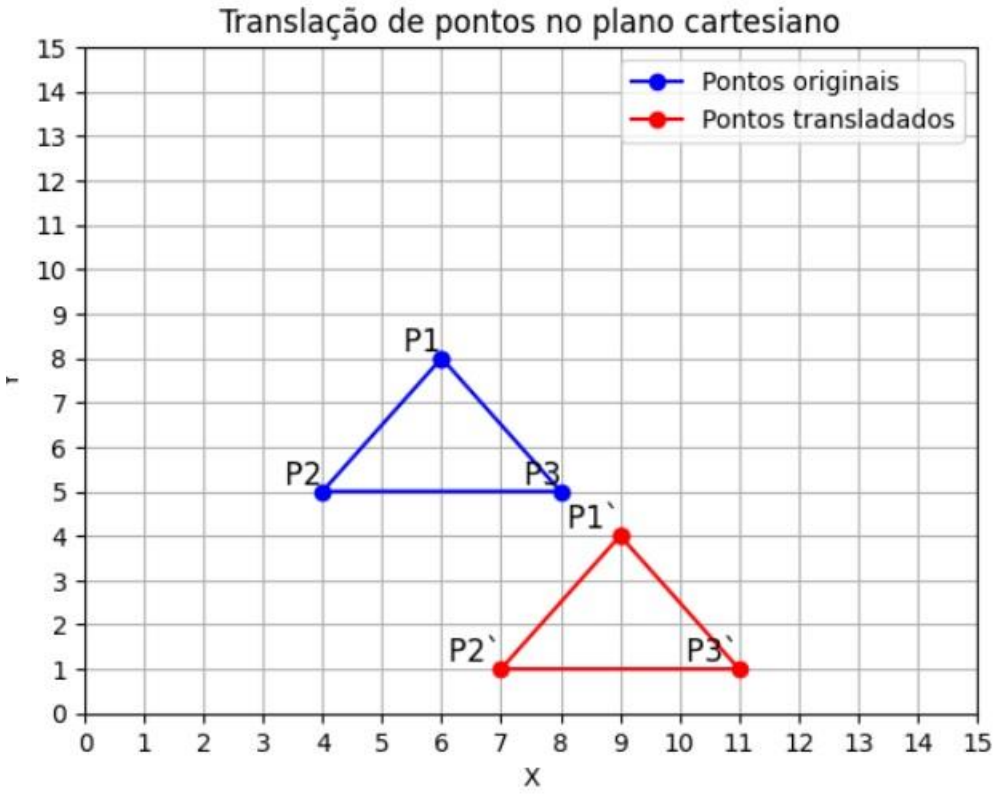
Translação

Vamos aplicar essa teoria na prática, através de um algoritmo em Python para a operação de translação para os pontos dos exemplos anteriores, slide 8 com 3 pontos.

Gráfico gerado:

```
# Define os pontos originais
p1 = (6, 8)  # Primeiro ponto
p2 = (4, 5)  # Segundo ponto
p3 = (8, 5)  # Terceiro ponto

# Define o vetor de translação
Tx = 3  # Translação no eixo X
Ty = -4  # Translação no eixo Y
```

p1'(?,?) = (9,4)
p2'(?,?) = (7,1)
p3'(?,?) = (11,1)



Escala

- É a alteração do **tamanho do objeto.**
- Aplicam-se as seguintes equações sobre os pontos do objeto:

$$x_u = x_o * S_x$$

$$y_u = y_o * S_y$$

- Onde S_x e S_y são os fatores de escala em relação aos eixos X e Y;
- A escala pode mudar a posição do objeto, pois ela sempre usa como referência a origem (0,0).

Escala

- Observações:
  - Se **S > 1** □ aumenta o tamanho do objeto;
  - Se **0 < S < 1** □ diminui o tamanho do objeto;
  - Se **S < 0** □ inverte/espelha o objeto.

o **S** representa o **Fator de Escala** (*Scaling Factor*). Como a operação de escala é uma multiplicação (P' = P.S), o número 1 é a fronteira que define o comportamento do objeto, representa 100% do tamanho original, P'=resultado da transformação, P=ponto original).

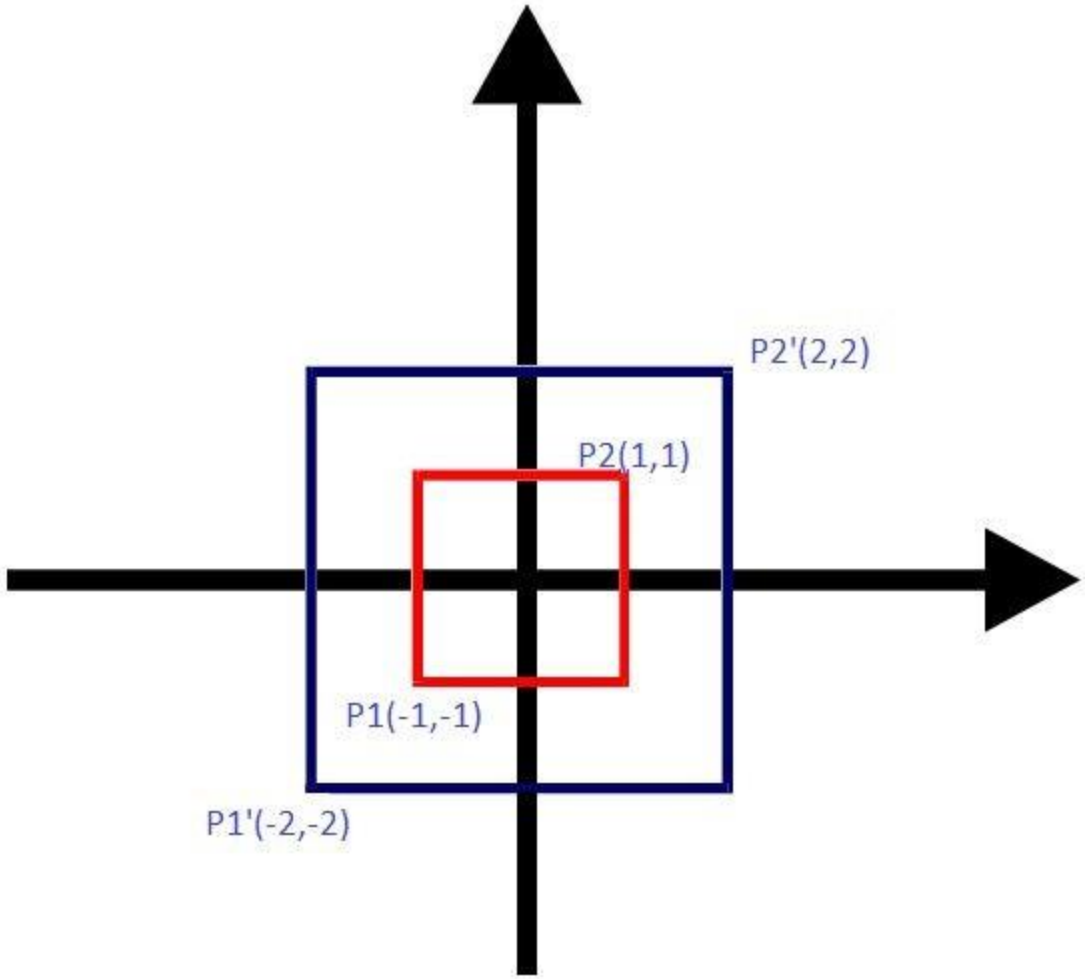
O 'S' é o multiplicador que aplicamos às coordenadas dos vértices. Em jogos, geralmente temos Sx (escala no eixo X) e Sy (escala no eixo Y).Se S = 2: O objeto dobra de tamanho (200%).
Se S = 0.5: O objeto fica pela metade (50%).

O 1 representa 100% do tamanho original.

- S > 1 (Aumento): Qualquer número maior que 1 "estica" o objeto. No motor de jogo (Unity ou Unreal), se colocar 'Scale=1.2', o personagem cresce 20%.
- 0 < S < 1 (Diminuição): Qualquer número entre 0 e 1 "espreme" o objeto. É uma multiplicação por fração.
- S = 1: O objeto mantém o tamanho original

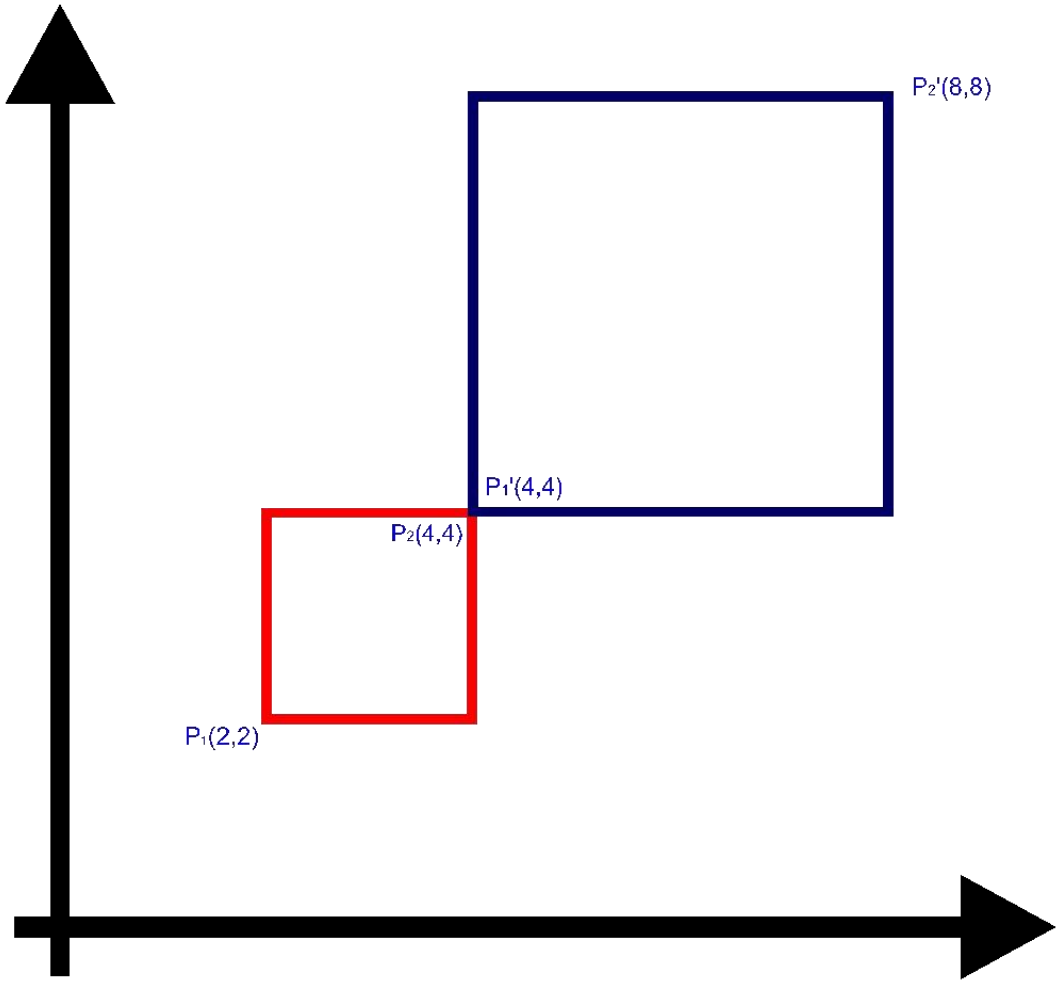
Escala

- Exemplo 1:
  - $P_1(-1,-1)$ e $P_2(1,1)$
  - $S_x = 2$ e $S_y = 2$
  - $P_1'(-2,-2)$ e $P_2'(2,2)$



Escala

- Exemplo 2:
  - $P_1(2,2)$ e $P_2(4,4)$
  - $S_x = 2$ e $S_y = 2$
  - $P_1'(4,4)$ e $P_2'(8,8)$



Escala

- A escala também pode ser representada na forma matricial através de uma **multiplicação de matrizes**:

$$P_o = \begin{bmatrix} x_o \\ y_o \end{bmatrix} \qquad P_u = \begin{bmatrix} x_u \\ y_u \end{bmatrix} \qquad S = \begin{bmatrix} S_x & 0 \\ 0 & S_Y \end{bmatrix}$$

$$P_u = S * P_o$$

$$P_u = \begin{bmatrix} S_x & 0 \\ 0 & S_Y \end{bmatrix} * \begin{bmatrix} x_o \\ y_o \end{bmatrix}$$

$$P_u = \begin{bmatrix} x_o S_x \\ y_o S_Y \end{bmatrix}$$

Escala

Vamos aplicar essa teoria na prática, através de um algoritmo em Python para a operação de escala para os pontos dos exemplos anteriores, slide 16. Exercício 03.

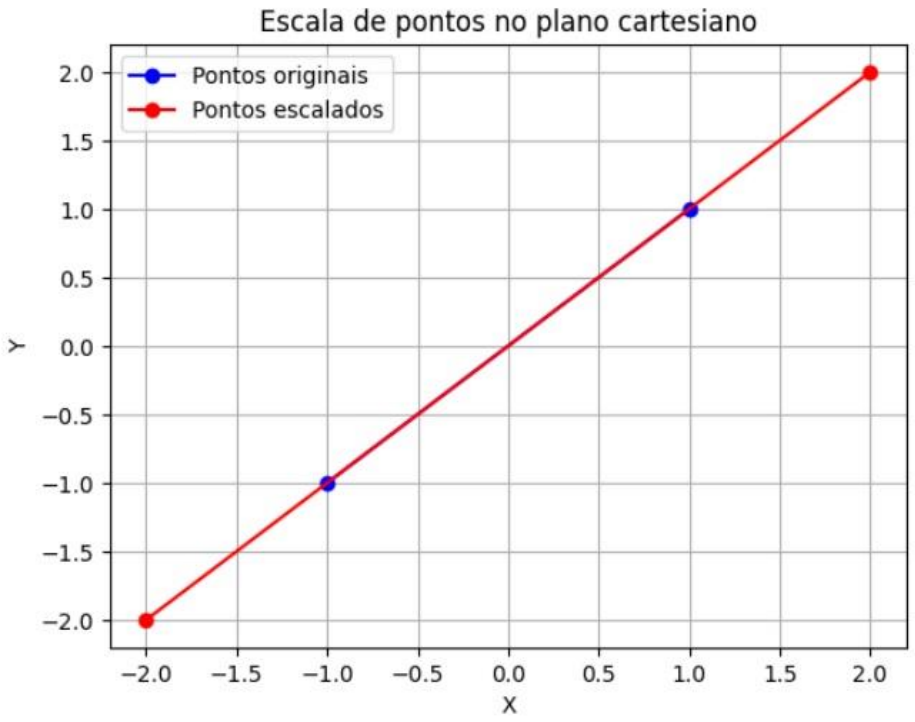
Agora adapte o algoritmo utilizado na translação para a operação de escala! Eles são bem parecidos, serão necessários algumas alterações! Só é preciso plotar os pontos, não precisa aquele quadrado em volta que esta descrito no slide 16.

OBS: Todos exercícios devem ser enviados, através de um link do seu repositório no github

Escala

Vamos aplicar essa teoria na prática, através de um algoritmo em Python para a operação de escala para os pontos dos exemplos anteriores, slide 16 e Slide 17. Exercício 03 a e b.

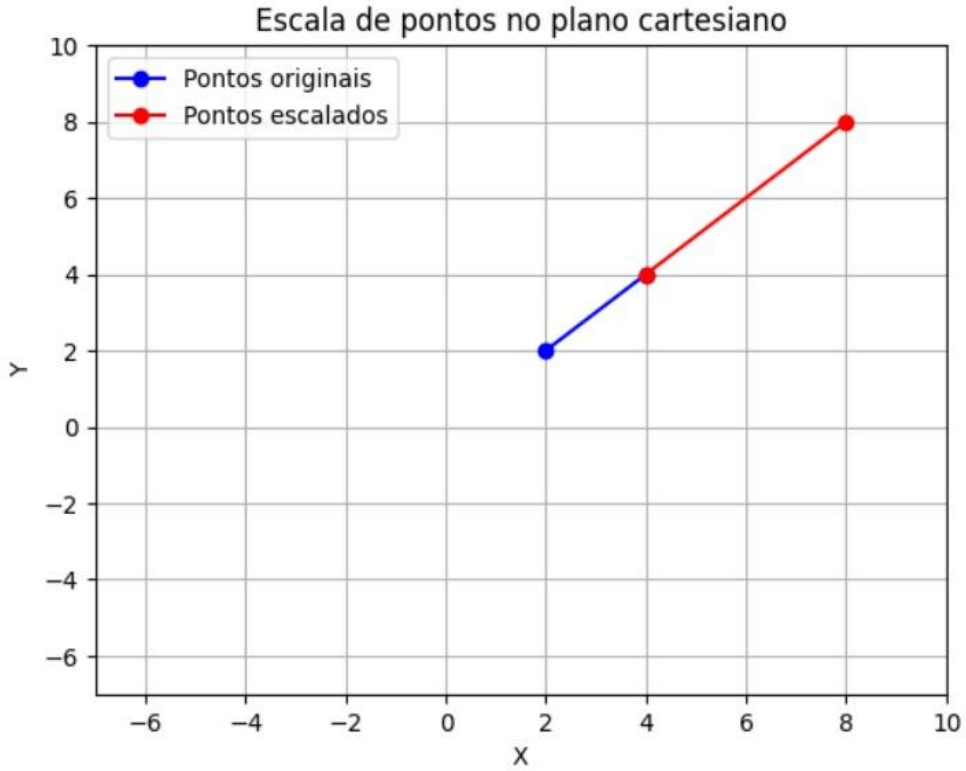
Gráficos gerados:



Exercício 03 a)

- $P_1(-1,-1)$ e $P_2(1,1)$
- $S_x = 2$ e $S_y = 2$
- $P_1'(-2,-2)$ e $P_2'(2,2)$

Exercício 03 b)



- $P_1(2,2)$ e $P_2(4,4)$
- $S_x = 2$ e $S_y = 2$
- $P_1'(4,4)$ e $P_2'(8,8)$

Escala

Exercício 03 a), testar os pontos para o exemplo 3b)

```python
import matplotlib.pyplot as plt


# Função para calcular a escala dos pontos
def escala(pontos, Sx, Sy):
    pontos_escala = []
    for ponto in pontos:
        x_u = ponto[0] * Sx
        y_u = ponto[1] * Sy
        pontos_escala.append((x_u, y_u))
    return pontos_escala

# Pontos originais
p1 = (-1, -1)
p2 = (1, 1)

# Fatores de escala
Sx = 2
Sy = 2

# Calcular a escala dos pontos
pontos_escala = escala([p1, p2], Sx, Sy)

print("Pontos escalados:")
for ponto in pontos_escala:
    print(ponto)
```

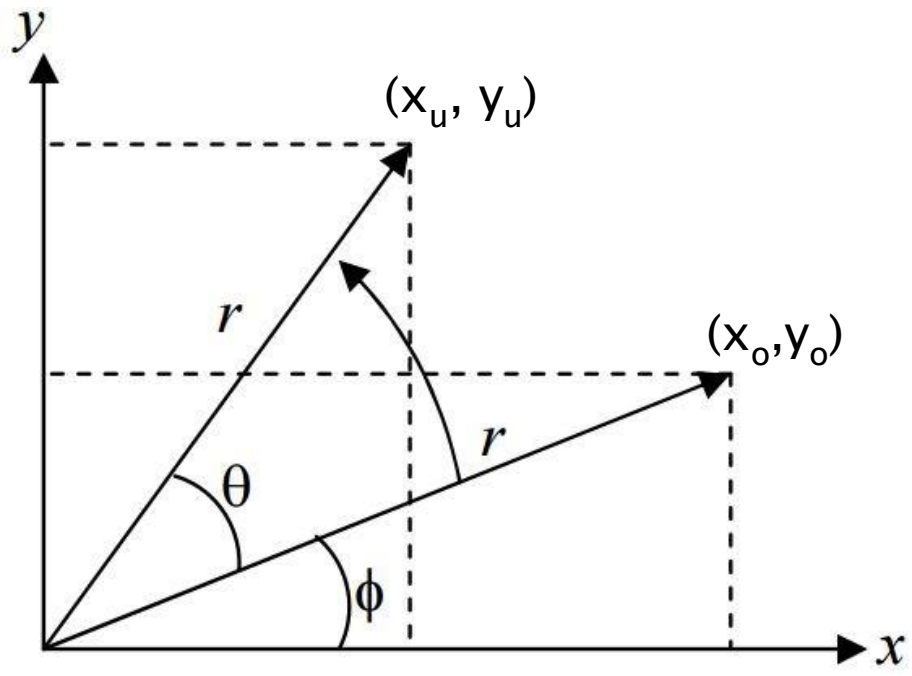
```python
# Plotar os pontos originais e os pontos escalados
plt.plot([p1[0], p2[0]], [p1[1], p2[1]], 'bo-', label='Pontos originais')

# 1. Primeiro, damos nomes claros aos pontos que a função criou
p1_esc = pontos_escala[0]  # Ponto 1 escalado
p2_esc = pontos_escala[1]  # Ponto 2 escalado

# 2. Agora criamos as listas para o gráfico usando nomes
lista_x = [ p1_esc[0], p2_esc[0] ]  # Pega o X do primeiro e o X do segundo
lista_y = [ p1_esc[1], p2_esc[1] ]  # Pega o Y do primeiro e o Y do segundo

# 3. O plot fica limpo
plt.plot(lista_x, lista_y, 'ro-', label='Pontos escalados')

# Definir os limites dos eixos
plt.xlim(-7, 10)  # Define o limite do eixo X de 0 a 15
plt.ylim(-7, 10)  # Define o limite do eixo Y de 0 a 15

# Configurações do gráfico
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Escala de pontos no plano cartesiano')
plt.grid(True)
plt.legend()
# Mostrar o gráfico
plt.show()
```

Rotação

- Modifica a **direção de um vetor** sobre algum eixo de rotação;
- No plano 2D, rotacionar um objeto significa **reposicionar** o mesmo em um **caminho circular**, semelhante aos ponteiros de um relógio;
  - O eixo de rotação neste caso é um vetor perpendicular ao plano xy e passa pelo centro de rotação □ como se fosse o eixo z;
- A rotação é especificada por um ângulo θ:
  - Valores positivos de θ rotacionam no sentido anti-horário;
  - **O centro de rotação é sempre a origem do sistema de coordenadas.**

Rotação

- Rotação de um vetor da posição (x_o,y_o) para a posição (x_u, y_u) segundo um ângulo θ relativo à origem do sistema de coordenadas



Rotação

- Define a orientação do objeto no universo;

- As equações de rotação de um ponto (x_o, y_o) de um ângulo são:

$$\boxed{x_u = x_o * \cos(\theta) - y_o * sen(\theta)}$$

$$\boxed{y_u = y_o * \cos(\theta) + x_o * sen(\theta)}$$

- A rotação sempre acontece na origem, assim o centro de rotação é o ponto 0,0.

Rotação

- Exemplo 1:
  - $P_1(2,2)$ e $P_2(4,4)$
  - Ângulo 45 graus

  - P_1:

    $$x_u = 2 * \cos(45) - 2 * sen(45) = 2 * 0.7071 - 2 * 0.7071 = 0$$

    $$y_u = 2 * \cos(45) + 2 * sen(45) = 2 * 0.7071 + 2 * 0.7071 = 2.8284$$

  - P_2:

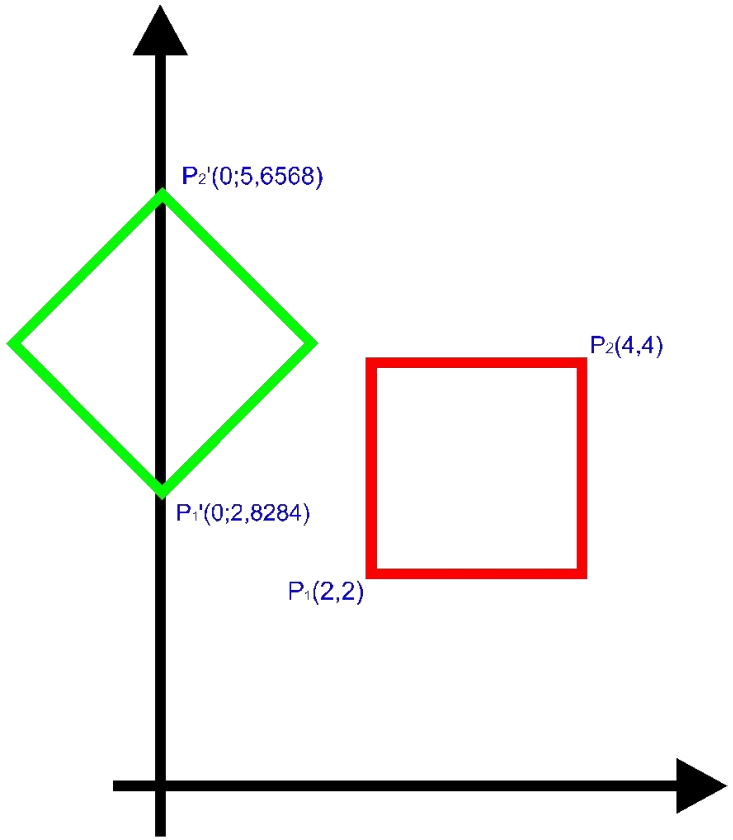
    $$x_u = 4 * \cos(45) - 4 * sen(45) = 4 * 0.7071 - 4 * 0.7071 = 0$$

    $$y_u = 4 * \cos(45) + 4 * sen(45) = 4 * 0.7071 + 4 * 0.7071 = 5.6568$$

  P_1'(0;2.8284) e P_2'(0;5.6568)

Rotação

- Exemplo 1:



Rotação

- A rotação também pode ser representada na forma matricial através de uma **multiplicação de matrizes**:

$$P_o = \begin{bmatrix} x_o \\ y_o \end{bmatrix} \qquad P_u = \begin{bmatrix} x_u \\ y_u \end{bmatrix} \qquad \mathrm{R} = \begin{bmatrix} \cos\theta & -sen\theta \\ sen\theta & \cos\theta \end{bmatrix}$$

$$P_u = R * P_o$$

$$P_u = \begin{bmatrix} \cos\theta & -sen\theta \\ sen\theta & \cos\theta \end{bmatrix} * \begin{bmatrix} x_o \\ y_o \end{bmatrix}$$

$$P_u = \begin{bmatrix} x_o \cos\theta - y_o sen\theta \\ x_o sen\theta + y_o \cos\theta \end{bmatrix}$$

Rotação

Exercício 04.

Agora adapte o algoritmo utilizado na translação para a operação de rotação! Eles são bem parecidos, serão necessários algumas alterações! Uma dica é que devemos receber (pontos, ângulo) como parâmetros. Use o exemplo do slide 25 para rotacionar P1(2,2) e P2(4,4).

Dica:

```
# Converter ângulo de graus para radianos (funções pyhton usam radianos
    angulo_rad = np.radians(angulo)
    # Calcular as coordenadas após a rotação
    xu = x0 * np.cos(angulo_rad) - y0 * np.sin(angulo_rad)
   .......
```

OBS: Todos exercícios devem ser enviados, através de um link do seu repositório no github

Rotação

Exercício 04.

```python
import numpy as np  # Importa a biblioteca NumPy para operações matemáticas
import matplotlib.pyplot as plt  # Importa a biblioteca Matplotlib para plotar gráficos

# Função para calcular a rotação dos pontos
def rotacao(pontos, angulo):
    pontos_rotacionados = []  # Lista para armazenar os pontos rotacionados
    angulo_rad = np.radians(angulo)  # Converter ângulo de graus para radianos

    for ponto in pontos:  # Percorre cada ponto na lista
        x0, y0 = ponto  # Obtém as coordenadas do ponto
        # Calcular as coordenadas após a rotação
        xu = round(x0 * np.cos(angulo_rad) - y0 * np.sin(angulo_rad), 2)
        yu = round(x0 * np.sin(angulo_rad) + y0 * np.cos(angulo_rad), 2)
        pontos_rotacionados.append((xu, yu))  # Adiciona o ponto rotacionado à lista

 return pontos_rotacionados  # Retorna a lista de pontos rotacionados

# Pontos originais
p1 = (2, 2)  # Primeiro ponto
p2 = (4, 4)  # Segundo ponto

# Ângulo de rotação (em graus)
angulo = 45  # Define o ângulo de rotação

# Calcular a rotação dos pontos
pontos_rotacionados = rotacao([p1, p2], angulo)
```

```python
# Imprimir pontos originais e rotacionados para verificação
print("Pontos originais:")
print(f"P1: {p1}")
print(f"P2: {p2}")

print("\nPontos rotacionados:")
for i, p in enumerate(pontos_rotacionados, start=1):
    print(f"P{i}`: {p}")

# Plotar os pontos originais e os pontos rotacionados
plt.plot([p1[0], p2[0]], [p1[1], p2[1]], 'bo-', label='Pontos originais')
plt.plot([p[0] for p in pontos_rotacionados], [p[1] for p in pontos_rotacionados], 'ro-', label='Pontos
rotacionados')  # Plota os pontos rotacionados em vermelho

# Adicionar rótulos aos pontos originais
plt.text(p1[0], p1[1], ' P1', fontsize=12, verticalalignment='bottom', horizontalalignment='right')
plt.text(p2[0], p2[1], ' P2', fontsize=12, verticalalignment='bottom', horizontalalignment='right')

# Adicionar rótulos aos pontos rotacionados
for i, p in enumerate(pontos_rotacionados, start=1):
    plt.text(p[0], p[1], f' P{i}`', fontsize=12, verticalalignment='bottom',
horizontalalignment='right')

# Configurações do gráfico
plt.xlabel('X')  # Define o rótulo do eixo X
plt.ylabel('Y')  # Define o rótulo do eixo Y
plt.title('Rotação de pontos no plano cartesiano')  # Define o título do gráfico
plt.grid(True)  # Exibe a grade no gráfico
plt.legend()  # Adiciona a legenda


# Mostrar o gráfico
plt.show()  # Exibe o gráfico
```

Matrizes de Transformação

- Matriz de Translação (soma):

$$T = \begin{bmatrix} T_x \\ T_Y \end{bmatrix}$$

- Matriz de Escala (multiplicação):

$$S = \begin{bmatrix} Sx & 0 \\ 0 & Sy \end{bmatrix}$$

- Matriz de Rotação (multiplicação):

$$\mathrm{R} = \begin{bmatrix} \cos\theta & -sen\theta \\ sen\theta & \cos\theta \end{bmatrix}$$

Coordenadas homogêneas

- Todas as transformações anteriores, **exceto** a **translação**, podem ser executadas por meio de **multiplicação de matrizes**;
- Em muitos casos precisaremos representar **combinações de várias transformações** em uma única matriz;
- Pelo fato de que a translação estará presente em quase todas as transformações, o ideal seria que ela também pudesse ser representada por uma **multiplicação de matrizes**;
- Para isso introduziu-se o conceito de **coordenadas homogêneas**.

Coordenadas homogêneas

- O conceito de coordenadas homogêneas consiste em adicionar uma dimensão a mais na matriz de transformação;

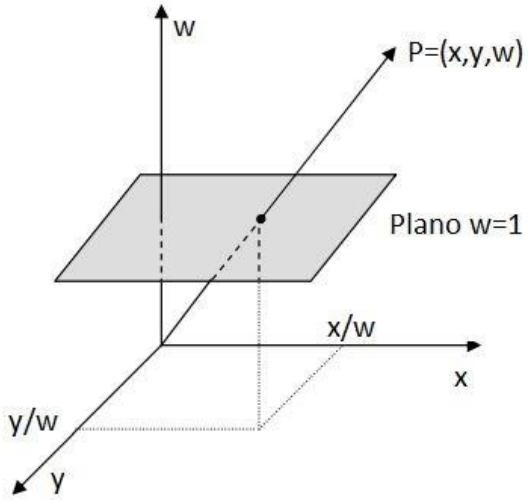
- Desta forma, um ponto 2D é representado por (x,y,w). Caso o valor de w não seja 1, deve-se dividir os componentes x e y por w:

  - Desta forma, os pontos (2, 3, 6) e (4, 6, 12) representariam a mesma informação:
    - Processo denominado homogeneização;
    - Neste caso, $\frac{x}{w}$ e $\frac{y}{w}$ são as coordenadas cartesianas do ponto homogêneo.

$$\begin{pmatrix} x & y & w \end{pmatrix} = \begin{pmatrix} \frac{x}{w} & \frac{y}{w} & 1 \end{pmatrix}$$

Coordenadas homogêneas

- Esta terceira coordenada do ponto pode ser interpretada geometricamente como um plano h=1 no R^3, conforme figura abaixo:



- O ponto $P' = (\frac{x}{w}, \frac{y}{w})$ é a projeção do ponto P(x, y, w) no plano w=1.

- Desta forma, qualquer múltiplo do ponto P homogêneo representa o mesmo ponto.

O "truque" da Computação Gráfica: Para representar um ponto de **2D (R^2)**, nós o "projetamos" em um espaço de **3D (R^3)**. Imagine que você tem uma folha de papel (X, Y) flutuando exatamente na altura **Z = 1** de uma sala. Esse "Z" é o nosso **h** (ou **w**) das coordenadas homogêneas.

Ex: imaginem que o componente w é como o zoom de uma lanterna. Se w=1, o ponto está no tamanho real. Se w aumenta, o ponto 'se afasta' e fica menor porque dividimos as coordenadas por ele. Isso é o que permite que a matemática entenda o que está perto e o que está longe na tela

Coordenadas Homogêneas

- Com o uso de coordenadas homogêneas, as transformações são definidas como:
  - **Translação**: $P_u = TP_o$
    - $P_u = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_o \\ y_o \\ 1 \end{bmatrix}$
  - **Escala**: $P_u = SP_o$
    - $P_u = \begin{bmatrix} S_X & 0 & 0 \\ 0 & S_Y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_o \\ y_o \\ 1 \end{bmatrix}$
  - **Rotação**: $P_u = RP_o$
    - $P_u = \begin{bmatrix} cos\theta & -sen\theta & 0 \\ sen\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_o \\ y_o \\ 1 \end{bmatrix}$

Combinações de transformações

- Quando desejamos aplicar **várias transformações** a um ou mais pontos, ao invés de aplicarmos uma transformação por vez, podemos **combiná-las em uma única matriz**, e então aplicar esta matriz resultante aos pontos de um objeto:
  - Isso reduz muito o custo computacional!
- O processo de **concatenação de matrizes** é feito por meio da multiplicação das matrizes que representam cada transformação:
  - É importante lembrar que a ordem em que as multiplicações são feitas faz diferença.
  - É associativa, porém não comutativa!
    - $AB \neq BA$ (não comutativa)
    - $A(BC) = (AB)C$ (associativa)
    - $(FG)^T = G^T F^T$ T=(Transposta)

Combinações de transformações

- Ex: AB≠BA (não comutativa)

A multiplicação de matrizes não é comutativa, ou seja, a ordem das multiplicações importa.

Matriz A:

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 0 & 1 & 4 \\ 5 & 2 & 1 \end{bmatrix}$$

Matriz B:

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Resultado de AB:

$$AB = \begin{bmatrix} 27 & 28 & 39 \\ 32 & 32 & 42 \\ 20 & 18 & 36 \end{bmatrix}$$

Resultado de BA:

$$BA = \begin{bmatrix} 14 & 8 & 6 \\ 33 & 10 & 27 \\ 35 & 18 & 39 \end{bmatrix}$$

Exemplo prático:

•Se aplicarmos **primeiro uma rotação** e **depois uma translação**, o objeto será rotacionado primeiro e depois deslocado.

•Se **primeiro transladarmos** e **depois rotacionarmos**, o objeto será deslocado antes da rotação, o que pode mudar completamente sua posição final!

Ou seja, **translação seguida de rotação** não é igual a **rotação seguida de translação**.

Combinações de transformações

- Ex: A(BC) = (AB)C (associativa)

A multiplicação de matrizes é **associativa**, ou seja, a ordem do agrupamento não altera o resultado.

Matrizes utilizadas:

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 0 & 1 & 4 \\ 5 & 2 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad C = \begin{bmatrix} 3 & 1 & 2 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix}$$

Cálculos:

1. **Multiplicamos B e C:**

$$BC = \begin{bmatrix} 14 & 12 & 16 \\ 20 & 18 & 30 \\ 36 & 36 & 50 \end{bmatrix}$$

2. **Multiplicamos A pelo resultado:**

$$A(BC) = \begin{bmatrix} 199 & 176 & 189 \\ 222 & 202 & 212 \\ 166 & 132 & 146 \end{bmatrix}$$

---

3. **Multiplicamos A e B primeiro:**

$$AB = \begin{bmatrix} 27 & 28 & 39 \\ 32 & 32 & 42 \\ 20 & 18 & 36 \end{bmatrix}$$

4. **Multiplicamos o resultado por C:**

$$(AB)C = \begin{bmatrix} 199 & 176 & 189 \\ 222 & 202 & 212 \\ 166 & 132 & 146 \end{bmatrix}$$

A multiplicação de matrizes **é associativa** (A(BC) = (AB)C), ou seja, podemos **agrupar** as transformações sem alterar o resultado.

Exemplo prático:

•Se queremos aplicar três transformações **T1, T2, T3**, podemos multiplicar **T2 e T3 primeiro** e depois aplicar **T1**:

- T1 * (T2 * T3)

•Ou podemos multiplicar **T1 e T2 primeiro** e depois aplicar **T3**:

- (T1 * T2) * T3

O resultado final será o mesmo, independentemente de como agrupamos as transformações.

Combinações de transformações

- Ex: $(AB)^T = B^T A^T$ T=(Transposta)

A transposição do produto de duas matrizes segue a regra:

$$(AB)^T = B^T A^T$$

Matrizes utilizadas:

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 0 & 1 & 4 \\ 5 & 2 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Cálculos:

1. Calculamos AB:

$$AB = \begin{bmatrix} 27 & 28 & 39 \\ 32 & 32 & 42 \\ 20 & 18 & 36 \end{bmatrix}$$

2. Transpomos AB:

$$(AB)^T = \begin{bmatrix} 27 & 32 & 20 \\ 28 & 32 & 18 \\ 39 & 42 & 36 \end{bmatrix}$$

---

3. Transpomos B e A separadamente:

$$B^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 0 & 8 \\ 3 & 6 & 9 \end{bmatrix}, \quad A^T = \begin{bmatrix} 2 & 0 & 5 \\ 1 & 1 & 2 \\ 3 & 4 & 1 \end{bmatrix}$$

4. Multiplicamos $B^T A^T$:

$$B^T A^T = \begin{bmatrix} 27 & 32 & 20 \\ 28 & 32 & 18 \\ 39 & 42 & 36 \end{bmatrix}$$

A transposição de uma matriz de transformação afeta a ordem das operações. Em gráficos computacionais, isso pode ser útil para encontrar transformações inversas ou refletir objetos.

Exemplo prático:
Se tivermos uma transformação F seguida de G, a transposição do resultado equivale a inverter a ordem das transformações transpostas:

•(F * G)^T = G^T * F^T

Combinações de transformações

- Exemplo: concatenação de uma **translação seguida de uma rotação:**
  - Primeiro obtemos a **matriz de transformação (M_T)**, multiplicando a matriz de rotação (R) pela de translação (T);
    - $\boldsymbol{M_T = RT}$ → a ordem importa!
    - $$M_T = \begin{bmatrix} cos\theta & -sen\theta & 0 \\ sen\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix}$$
    - $$\boxed{M_T = \begin{bmatrix} cos\theta & -sen\theta & T_x cos\theta - T_y sen\theta \\ sen\theta & cos\theta & T_x sen\theta + T_y cos\theta \\ 0 & 0 & 1 \end{bmatrix}}$$

Combinações de transformações

- Exemplo: concatenação de uma **translação seguida de uma rotação:**
  - Em seguida, multiplicamos a **matriz de transformação** ($\boldsymbol{M_T}$) pelo ponto $\mathbf{P_o}$;
    - $\boldsymbol{P_u} = \boldsymbol{M_T}\boldsymbol{P_o}$
    - $$\boxed{P_u = \begin{bmatrix} cos\theta & -sen\theta & T_x cos\theta - T_y sen\theta \\ sen\theta & cos\theta & T_x sen\theta + T_y cos\theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_o \\ y_o \\ 1 \end{bmatrix}}$$

Referências e material de apoio

AZEVEDO, Eduardo; CONCI, Aura. **Computação gráfica:** teoria e prática. 6ª tiragem 2003. Rio de Janeiro, RJ: Elsevier, 2003. 353 p.
FOLEY, James D. **Computer graphics:** principles and practice. 2nd. ed. Massachusetts: Addison-Wesley, 1997. 1175 p.
HETEM JUNIOR, Annibal. **Computação gráfica.** Rio de Janeiro, RJ: LTC, 2006. 161 p. (Coleção Fundamentos de Informática).

COHEN, Marcelo; MANSSOUR, Isabel Harb. **OpenGL:** uma abordagem prática e objetiva. São Paulo, SP: Novatec, 2006. 478 p.
GOMES, Jonas; VELHO, Luiz. **Computação gráfica.** Rio de Janeiro, RJ: IMPA, 1998. V 1. (Série de computação e matemática).
HEARN, Donald; BAKER, M. Pauline. **Computer grafhics:** C version. 2. ed. London: Prentice - Hall, 1997. 352 p.
HILL JR., F. S.; X HILL JÚNIOR, Francis S. **Computer graphics using open GL.** 2nd. ed. New Jersey: Prentice Hall, c2001. 922 p.
WATT, Alan. **3D computer graphics.** 3nd. ed. Harlow: Addison-Wesley, 2000. 570 p.
Material do Professor Guilherme Chagas Kurtz, 2023.

Obrigado pela atenção!!

Email: andre.flores@ufn.edu.br

Santa Maria – RS 2026